

LECTURE 11

Data Modeling and Database Design

From a fuzzy business conversation to precise tables, columns, and keys

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

What This Lecture Covers

- 1 Why data modeling comes before building anything, not after
- 2 Conceptual data models — capturing meaning, independent of technology
- 3 Logical data models — adding relational structure, still DBMS-independent
- 4 Relational database design as this course's specific focus
- 5 Mapping conceptual → logical → relational schema
- 6 Practical considerations that separate a good design from a merely working one

Why Model Before You Build

- A data model is a shared vocabulary for data, its relationships, and its constraints — letting a designer, a client, and a DBMS agree on what “a branch” or “a viewing” means before any SQL is written.
- Tables are a technology-specific answer to a business question — separating “what does the business need?” from “how do we implement it?” lets you get the meaning right first, cheaply.

WARNING

The cost of getting this wrong compounds: a missing requirement caught during modeling costs a conversation and a redrawn diagram. The same requirement, caught after production data depends on the shape, can cost a multi-week migration.

Conceptual → Logical → Relational Schema

Conceptual model

“What are the things, and how do they relate?” — technology-independent, for business stakeholders



Logical model

“What tables, columns, and keys does this need?” — DBMS-product-independent, for designers



Relational schema

“What does CREATE TABLE actually say?” — concrete data types and DBMS-specific detail

Conceptual Data Models

- A technology-independent description of an organization's data: the entities (things to track), relationships (connections), and attributes (properties).
- No mention of tables, columns, data types, or which DBMS will run it.
- The Entity-Relationship (ER) model, next lecture's subject, is the standard notation for building conceptual models.

“A branch employs several staff.” “A client may view several properties, and a property may be viewed by several clients.” “Each property is owned by exactly one private owner.”

TIP

A conceptual model should survive a change of DBMS unchanged. If moving from PostgreSQL to Oracle forces you to redraw it, the model was capturing implementation detail, not business meaning.

Logical Data Models

- Takes the entities, relationships, and attributes from the conceptual model and organizes them according to the rules of a particular family of data model — relational, hierarchical, network, or object-oriented.
- This course works entirely within the relational family: a logical model means a complete set of relation schemas — names, attributes, primary keys, foreign keys — with no data types or storage details yet.
- Answers “what relations do we need, and how do they reference each other?” — not “is salary a DECIMAL(8,2) or an INT?”

Three Properties of a Good Design

1

Correctness

The design faithfully represents every entity, relationship, and business rule the organization actually has — no more, no less.

2

Non-redundancy

The same fact is not stored in more than one place, so it can never drift out of sync (the motivation for normalization, Unit 5).

3

Robustness under real use

The design supports the queries, transactions, and volume of data the organization will actually throw at it.

WARNING

Relational database design is not the same as “writing SQL.” Syntactically perfect CREATE TABLE statements can still represent the business incorrectly — SQL syntax accepts it without complaint; design discipline catches it anyway.

Conceptual Model → Logical Model

- Every strong entity type becomes a relation; the entity's identifying attribute becomes the primary key.
- Every simple attribute becomes a column of that relation.
- Every relationship type becomes a foreign key (1:1, 1:N) or an entirely new relation (M:N) — the precise rule depends on cardinality, covered fully in Lecture 12.

Branch
<u>branchNo</u>
street
city
postcode
Conceptual — an ER entity type

Logical — the mapped relation

branchNo	street	city	postcode
B003	163 Main St	Lahore	54000
B005	22 Deer Rd	Karachi	74200

Nothing was lost — every attribute became a column and the identifying attribute became the primary key — but nothing was added either: no data type, no index, no storage engine.

Logical Model → Relational Schema

- Concrete data types for every attribute — DECIMAL(8,2), INTEGER, or NUMERIC?
- Column-level constraints — NOT NULL, CHECK constraints enforcing domain rules.
- Indexing strategy — primary keys and foreign keys, at minimum.
- Naming conventions and DBMS-specific CREATE TABLE syntax.

```
CREATE TABLE Branch (  
    branchNo  CHAR(4)      NOT NULL,  
    street    VARCHAR(40)  NOT NULL,  
    city      VARCHAR(30)  NOT NULL,  
    postcode  CHAR(5)      NOT NULL,  
    PRIMARY KEY (branchNo)  
);
```

NOTE

Connolly's textbook calls this final translation step physical database design — unlike the logical model, it is allowed, even expected, to differ between DBMS products and trade conceptual purity for performance.

Relational Database Design Considerations

Consideration	Question to Ask
Redundancy	Is any fact stored in more than one place?
Update anomalies	Can inserting, updating, or deleting one row corrupt or lose other facts?
Appropriate keys	Is the primary key stable and minimal?
Referential integrity	Does every foreign key correctly reference a real, enforced primary key?
Growth and volume	Will this design still perform acceptably at 100x the current data volume?
Security and access	Should every user see every column (e.g. salary)?
Naming consistency	Are attribute names consistent across every relation that shares a domain?

A Cheap Sanity Check

TIP

For every relation, read its name and attributes as a sentence: “A Branch has a branchNo, a street, a city, and a postcode.” If the sentence sounds natural and every attribute clearly belongs to that one thing, the relation is probably well-formed.

If you find yourself saying “...and also the name and address of whoever manages it” for a relation that isn't Staff, that's redundancy trying to sneak in — exactly what Unit 5's normalization theory formalizes and eliminates.

KEY TAKEAWAYS

Data Modeling and Database Design

- 1 Data modeling builds a shared, precise vocabulary before any technology commitment is made.
- 2 Conceptual model (technology-independent) → logical model (relational structure, DBMS-independent) → relational schema (concrete types, constraints, syntax).
- 3 Relational database design is judged by correctness, non-redundancy, and robustness under real use — not merely “does the SQL compile.”
- 4 Mapping conceptual → logical: entity → relation, attribute → column, relationship → foreign key or new relation depending on cardinality.
- 5 A syntactically correct schema can still be a poor design — always check redundancy, anomalies, keys, integrity, growth, and security.